Attention 学习资料

Attention 是现代 Transformer 架构中的核心机制之一。

Attention 的基本思想是：对于当前需要处理的信息，模型并不是平均关注所有输入，而是计算不同输入的重要程度，再根据重要程度进行加权。

Self-Attention

Self-Attention 表示 Query、Key 和 Value 都来自同一个输入序列。

假设输入经过线性映射得到 Q、K、V，则 Scaled Dot-Product Attention 可以表示为：

Attention(Q, K, V)
=
softmax(QK^T / sqrt(d_k))V

其中 Q 表示 Query，K 表示 Key，V 表示 Value。

Q 和 K 的点积用于衡量不同 token 之间的相关程度。

Scaled Dot-Product Attention

首先计算：

QK^T

它可以得到 Query 与各个 Key 之间的匹配程度。

随后除以：

sqrt(d_k)

这样做是为了避免维度较大时点积数值过大，从而使 Softmax 进入梯度较小的区域。

然后经过 Softmax：

softmax(QK^T / sqrt(d_k))

得到注意力权重。

最后用这些权重对 V 进行加权求和，得到 Attention 输出。

Causal Mask

在 Decoder-Only 大语言模型中，通常还会加入 Causal Mask。

它的作用是防止当前位置看到未来 token。

因此模型在预测下一个 token 时，只能使用当前位置之前已经出现的信息。

Multi-Head Attention

Multi-Head Attention 会把 Attention 分成多个注意力头。

每一个 Head 都有自己的 Q、K、V 投影，可以从不同的表示子空间学习信息。

多个 Head 分别完成 Attention 计算后，会把输出进行拼接。

随后再经过一次线性投影，得到 Multi-Head Attention 的最终输出。

假设 hidden size 为 1024，一共有 8 个注意力头，则每个 Head 可以处理其中一部分表示空间。

Multi-Head Attention 的优势在于，不同 Head 可以关注不同类型的关系。

例如，有的 Head 可能更加关注相邻 token，有的 Head 可能更加关注长距离依赖关系。

Attention 与 Transformer

Transformer Block 通常由 Attention、Feed Forward Network、Residual Connection 和 Normalization 等部分共同组成。

Attention 负责 token 之间的信息交互，而 Feed Forward Network 则进一步处理每个 token 的表示。